College of Computer Science and Information Technology
Department of Computer Engineering

ARTI 404 – Image Processing

Lab 7(B) Report: Mathematical and Statistical Filters

Student Name Mohammed Omar Halawani

Student ID 2240001174

Academic Year 2025 – 2026

1. Objective

This lab adds four types of noise to an image (Gaussian, salt-and-pepper, exponential and uniform) and then removes noise with two families of spatial filters. The first family is the mean filters: arithmetic, geometric, harmonic and contra-harmonic. The second is the order-statistic filters: median, max, min and midpoint. Everything was implemented in Python with OpenCV, NumPy and Matplotlib.

2. Images Used

- **noise.jpg** – 508 × 506 synthetic test image, loaded as grayscale. It has a dark background (gray level ≈ 24), a mid-gray square (≈ 138) and a bright disk (≈ 232). Its flat regions make the noise and the effect of each filter easy to see.
- **salt&Pepper.jpg** – 225 × 224 image provided with the lab. The notebook does not use it.

3. Setup

```
import cv2
import numpy as np

from IPython.display import Image, display
from matplotlib import pyplot as plt

plt.rc('text', usetex=False)
```

`plt.imshow` is called without `vmin` and `vmax` in every figure, so each image is stretched to its own minimum and maximum. Brightness can therefore only be compared roughly between panels.

4. Adding Noise to an Image

The function `add_noise` creates each noise model:

- **Gaussian:** zero-mean normal noise with variance 0.01 ($\sigma = 0.1$) on the [0, 1] scale, which is $\sigma \approx 25.5$ gray levels.
- **Salt & pepper:** 4% of the pixels are corrupted, half set to 255 (salt) and half to 0 (pepper).
- **Exponential:** noise with scale 10 (mean 10) is added. It is always positive, so it also brightens the image.
- **Uniform:** noise drawn uniformly from [–20, 20].

```
# ========== ( Adding nosie to an Image ) =================

# 0. Read an Image
imgPath = 'noise.jpg'
img = cv2.imread(imgPath, 0)

# 1. Function that handle all noise algorithm (Guassian, Exp, Uniform & Salt&Pepper)
def add_noise(image, noise_type):
    row, col = image.shape

    if noise_type == "gaussian":
        mean = 0
        var = 0.01  # Adjust for more/less noise
        sigma = var**0.5
        gauss = np.random.normal(mean, sigma, (row, col))
        noisy = image / 255 + gauss
        return np.clip(noisy * 255, 0, 255).astype(np.uint8)

    elif noise_type == "salt_pepper":
        s_vs_p = 0.5
        amount = 0.04
        noisy = np.copy(image)
        # Salt (white pixels)
        num_salt = np.ceil(amount * image.size * s_vs_p)
        coords = [np.random.randint(0, i - 1, int(num_salt)) for i in image.shape]
        noisy[tuple(coords)] = 255
        # Pepper (black pixels)
        num_pepper = np.ceil(amount * image.size * (1.0 - s_vs_p))
        coords = [np.random.randint(0, i - 1, int(num_pepper)) for i in image.shape]
        noisy[tuple(coords)] = 0
        return noisy

    elif noise_type == "exponential":
        # Scale is the inverse of the rate (lambda)
        expo = np.random.exponential(scale=10.0, size=(row, col))
        noisy = image + expo
        return np.clip(noisy, 0, 255).astype(np.uint8)

    elif noise_type == "uniform":
        low = -20
        high = 20
        uni = np.random.uniform(low, high, (row, col))
        noisy = image + uni
        return np.clip(noisy, 0, 255).astype(np.uint8)

# 2. Load image in grayscale
img = cv2.imread(imgPath, 0)

# 3. Generate Noisy Images
noise_gauss = add_noise(img, "gaussian")
noise_sp = add_noise(img, "salt_pepper")
noise_expo = add_noise(img, "exponential")
noise_uni = add_noise(img, "uniform")

# 4. Display in 2x2 Grid
plt.figure(figsize=(12, 10))

plt.subplot(221), plt.imshow(noise_gauss, cmap='gray'), plt.title('Gaussian Noise')
```

```
plt.subplot(222), plt.imshow(noise_sp, cmap='gray'), plt.title('Salt & Pepper Noise')
plt.subplot(223), plt.imshow(noise_expo, cmap='gray'), plt.title('Exponential Noise')
plt.subplot(224), plt.imshow(noise_uni, cmap='gray'), plt.title('Uniform Noise')

for i in range(1, 5):
    plt.subplot(220 + i).axis('off')

plt.tight_layout()
plt.show()
```

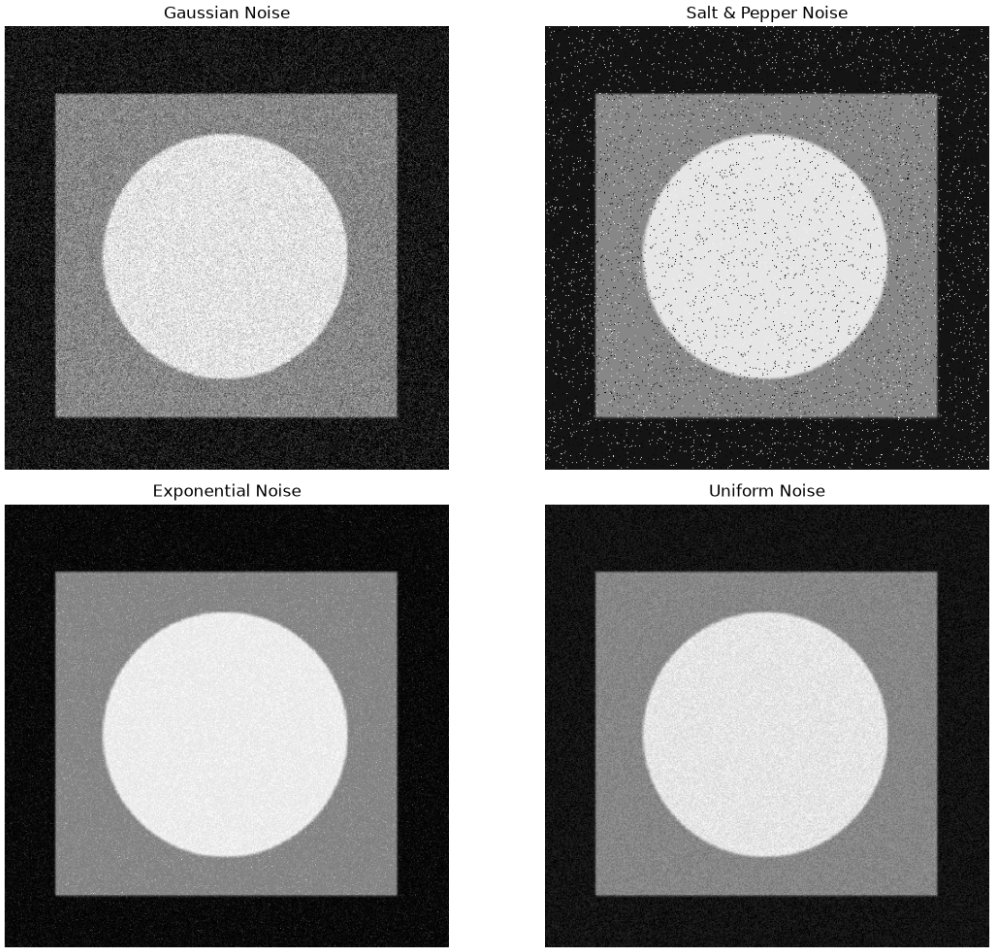


Figure 1: The test image corrupted with Gaussian, salt-and-pepper, exponential and uniform noise.

Discussion: Gaussian noise gives a fine grain over the whole image and is the strongest of the four. Salt-and-pepper noise gives isolated white and black dots, which stand out most where they contrast with the region: white dots in the dark background and black dots on the bright disk. Exponential and uniform noise look like a weaker grain. The noise is random, so the exact values change every run. In a re-run with the same parameters, the RMS error from the clean image was about 23 for Gaussian, 31 for salt-and-pepper, 13 for exponential and 12 for uniform noise. Exponential noise also raised the mean brightness by about 9 gray levels. Because the background is only about 24, about 18% of the background pixels are clipped to 0 when Gaussian noise is added. This is important for the filters below.

From this point on, every filter in the notebook is applied to `noise_gauss`, the Gaussian-noise image. The other noisy images are listed only in comments.

5. Arithmetic Mean Filter

The arithmetic mean filter replaces each pixel with the average of its m × n neighbourhood:

$$f(x, y) = (1 / mn) \Sigma g(s, t)$$

Method 1 uses OpenCV's built-in `cv2.blur` with a 3×3 window on the Gaussian-noise image:

```
# --- METHOD 1: Using Built-in OpenCV function ---

# 2. Apply Arithmetic Mean Filter (using a 5x5 blur window)
# The larger the numbers in (3, 3), the smoother the image becomes.,
# you can change kernel size to (5,5), (7,7) etc.
filtered_img = cv2.blur(noise_gauss, (3, 3))

display(filtered_img)
```

```
array([[ 7, 17, 18, ...,  7, 10, 10],
       [25, 26, 19, ..., 14, 18, 16],
       [29, 30, 16, ..., 15, 22, 23],
       ...,
       [28, 24, 25, ..., 20, 19, 18],
       [29, 23, 25, ..., 18, 18, 13],
       [31, 20, 29, ..., 18, 16, 12]], shape=(506, 508), dtype=uint8)
```

Method 2 builds the same 3×3 kernel of ones divided by 9 and applies it with `cv2.filter2D`:

```
# --- METHOD 2: Manual implementation using a Kernel ---

# An Arithmetic Mean kernel is just a matrix of 1s divided by the total number of pixels.
kernel_size = 3
kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size * kernel_size)
arithmetic_mean_manual = cv2.filter2D(img, -1, kernel)

display(arithmetic_mean_manual)
```

```
array([[24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       ...,
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24],
       [24, 24, 24, ..., 24, 24, 24]], shape=(506, 508), dtype=uint8)
```

Discussion: The notebook displays these results as NumPy arrays rather than plotting them, so there is no figure for this section. The two methods compute the same thing. On the same input, `cv2.blur` and `cv2.filter2D` with the normalized kernel give identical pixels. However, Method 2 is applied to `img`, the clean image, rather than to the noisy one. That is why its output is a constant 24 at the corners, which is the flat background value of the original image. In the re-run, the 3×3 mean filter reduced the RMS error of the Gaussian-noise image from about 23 to about 8. It does this by averaging the noise away, at the cost of slightly blurring the edges of the square and the disk.

6. Geometric Mean Filter

The geometric mean filter uses the product of the pixels in the window:

$$f(x, y) = [\, \Pi\, g(s, t)\,]^{1/mn}$$

The product of nine values can overflow, so the code computes it in the log domain: it takes the logarithm of each pixel, averages the logs with a box filter, and takes the exponent of the result. A small $\varepsilon = 10^{-6}$ is added to avoid log(0).

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def geometric_mean_filter(image, kernel_size=3):
    # 1. Convert to float64 and add a tiny epsilon to avoid log(0)
    img_float = image.astype(np.float64) + 1e-6

    # 2. Apply the log-transform
    log_img = np.log(img_float)

    # 3. Apply a box filter to the log image (calculates the average of logs)
    kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size * kernel_size)
    mean_log = cv2.filter2D(log_img, -1, kernel)

    # 4. Take the exponent to get back to the geometric mean
    geo_mean = np.exp(mean_log)

    # 5. Clip and convert back to 8-bit
    return np.clip(geo_mean, 0, 255).astype(np.uint8)

# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")

noise_img = noise_gauss

# Apply filter
result = geometric_mean_filter(noise_img, kernel_size=3)
```

```
# Display Comparison
plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Noisy Image')
plt.subplot(122), plt.imshow(result, cmap='gray'), plt.title('Geometric Mean Result')
plt.show()
```

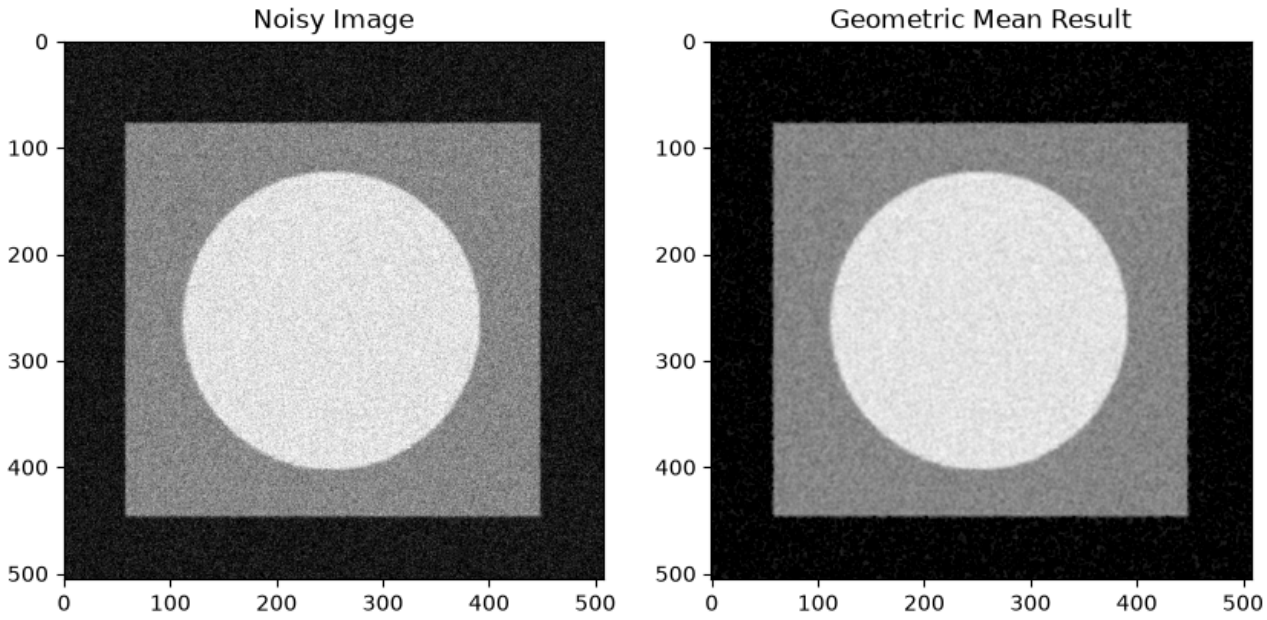


Figure 2: Gaussian-noise image and the result of the 3×3 geometric mean filter.

Discussion: Inside the square and the disk the noise is smoothed about as well as with the arithmetic mean (RMS error ≈ 9 compared with ≈ 8 in the re-run). The dark background, however, turns almost black. The geometric mean is very sensitive to small values: one pixel that was clipped to 0 becomes 10^{-6}, and its logarithm (≈ −13.8) pulls the whole window's result close to zero. Since about 18% of the noisy background pixels are 0, most background windows contain at least one zero. In the re-run, about half of the background pixels came out as 0 and the image became darker overall (mean −10 gray levels). The geometric mean works well for Gaussian noise in brighter regions but fails on pepper noise and on dark regions where noise is clipped.

7. Harmonic Mean Filter

$$f(x, y) = mn / \Sigma (1 / g(s, t))$$

The code computes the reciprocal of each pixel, sums the reciprocals over the window with a kernel of ones, and divides the window size by that sum.

```
def harmonic_mean_filter(image, kernel_size=3):
    # 1. Convert to float64 to avoid overflow during calculation
    # Add a tiny epsilon to handle black pixels (0)
    img_float = image.astype(np.float64) + 1e-6
```

```
    # 2. Calculate the reciprocal of every pixel (1/P)
    reciprocal_img = 1.0 / img_float

    # 3. Sum the reciprocals in a window and divide by window size
    # This is equivalent to applying a standard mean filter to the reciprocal image
    kernel = np.ones((kernel_size, kernel_size), np.float32)
    sum_reciprocal = cv2.filter2D(reciprocal_img, -1, kernel)

    # 4. Final step of the formula: (Window Size) / Sum of Reciprocals
    # Note: Window size is kernel_size * kernel_size
    n = kernel_size * kernel_size
    harmonic_mean = n / sum_reciprocal

    # 5. Clip the results and convert back to 8-bit
    return np.clip(harmonic_mean, 0, 255).astype(np.uint8)

# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")

noise_img = noise_gauss

# Apply the filter
cleaned_img = harmonic_mean_filter(noise_img, kernel_size=3)

# Display Comparison
plt.figure(figsize=(10, 5))
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Noisy Image')
plt.subplot(122), plt.imshow(cleaned_img, cmap='gray'), plt.title('Harmonic Mean Result')
plt.show()
```

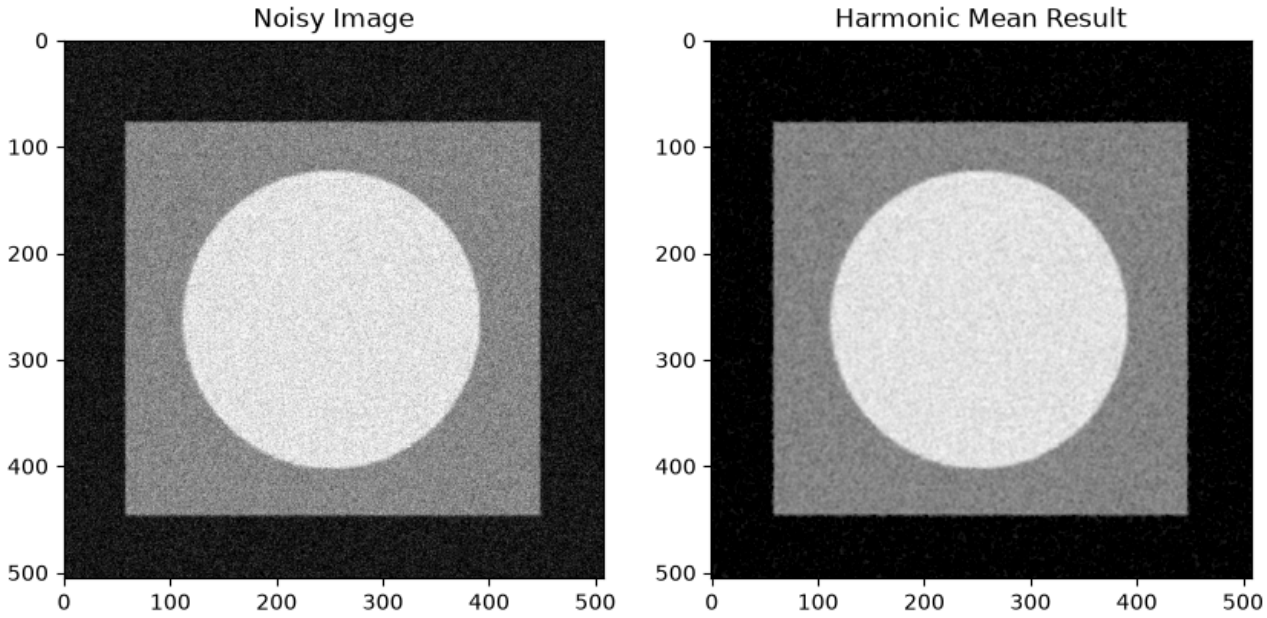


Figure 3: Gaussian-noise image and the result of the 3×3 harmonic mean filter.

Discussion: The result looks very similar to the geometric mean, with an even darker background. A zero pixel has a reciprocal of 10^6, which dominates the sum and drives the output to almost 0. In the re-run, about 83% of the background pixels became 0. The harmonic mean filter is known to work well for salt noise and Gaussian noise but to fail for pepper noise. Here the Gaussian noise clipped at 0 acts like pepper noise, which explains the result.

8. Contra-Harmonic Mean Filter

$$f(x, y) = \Sigma\, g(s, t)^{Q+1} \,/\, \Sigma\, g(s, t)^{Q}$$

The order Q controls the behaviour. Q = 0 gives the arithmetic mean and Q = −1 gives the harmonic mean. Positive Q gives more weight to bright pixels and removes pepper noise, and negative Q gives more weight to dark pixels and removes salt noise. The notebook uses Q = 1.5 and Q = −1.5.

```
def contra_harmonic_mean(image, kernel_size=3, Q=0):
    # 1. Convert to float to handle high powers and prevent overflow
    img_float = image.astype(np.float64) + 1e-6

    # 2. Calculate numerator: Sum of pixels to the power (Q + 1)
    numerator = np.power(img_float, Q + 1)
    kernel = np.ones((kernel_size, kernel_size), np.float32)
    num_sum = cv2.filter2D(numerator, -1, kernel)

    # 3. Calculate denominator: Sum of pixels to the power (Q)
    denominator = np.power(img_float, Q)
    den_sum = cv2.filter2D(denominator, -1, kernel)

    # 4. Divide and clean up
    # Handle potential division by zero
    result = num_sum / (den_sum + 1e-6)

    return np.clip(result, 0, 255).astype(np.uint8)

# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")

noise_img = noise_gauss

# --- APPLICATIONS ---
# Q > 0 (e.g., 1.5): Removes Pepper Noise (black spots)
pepper_removed = contra_harmonic_mean(noise_img, kernel_size=3, Q=1.5)

# Q < 0 (e.g., -1.5): Removes Salt Noise (white spots)
salt_removed = contra_harmonic_mean(noise_img, kernel_size=3, Q=-1.5)

# --- DISPLAY ---
plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(noise_img, cmap='gray'), plt.title('Original Noisy')
```

```
plt.subplot(132), plt.imshow(pepper_removed, cmap='gray'), plt.title('Q=1.5 (Removes
Pepper)')
plt.subplot(133), plt.imshow(salt_removed, cmap='gray'), plt.title('Q=-1.5 (Removes Salt)')
plt.show()
```

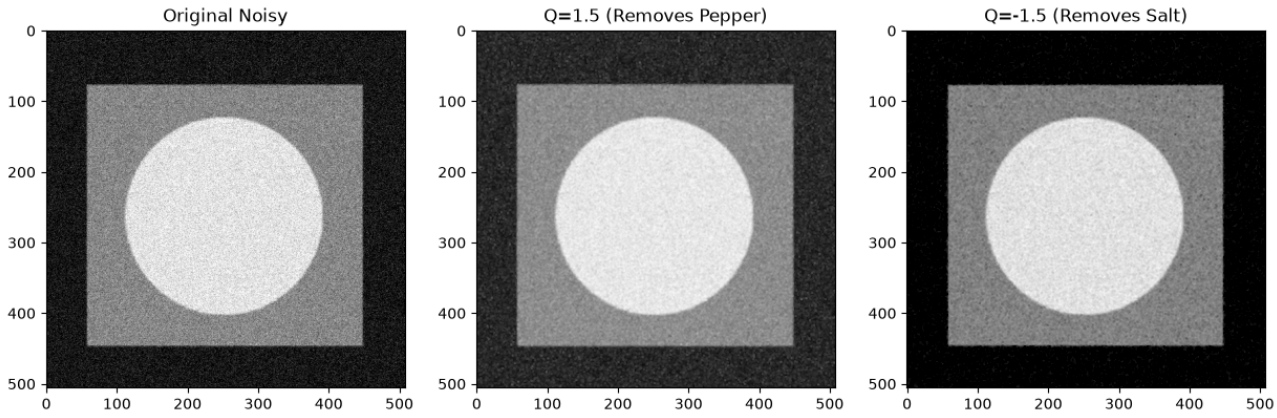


Figure 4: Gaussian-noise image and the contra-harmonic mean results for Q = 1.5 and Q = –1.5.

Discussion: Both results reduce the grain inside the square and the disk, but they move the brightness in opposite directions. With Q = 1.5 the bright pixels dominate each window, so the dark background is lifted to a lighter gray (about 45 instead of 24 in the re-run) and the image becomes brighter overall. With Q = –1.5 the dark pixels dominate, and the result behaves like the harmonic mean: the background becomes almost completely black. The titles "Removes Pepper" and "Removes Salt" describe what these settings do to salt-and-pepper noise. This notebook applies them to Gaussian noise, so the figure shows their bias rather than impulse-noise removal. Choosing the wrong sign of Q for the noise makes the result worse.

9. Median Filter

The median filter is an order-statistic filter. It sorts the values in the window and outputs the middle one. `cv2.medianBlur` requires an odd kernel size, and 3×3 is used here.

```
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")

noise_img = noise_gauss

# 2. Apply Median Filter
# The second argument is the 'ksize' (kernel size).
# It MUST be an odd integer (3, 5, 7, etc.)
median_result = cv2.medianBlur(noise_img, 3)

# 3. Compare Results
plt.figure(figsize=(12, 6))
```

```
plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Original (Salt & Pepper
Noise)'), plt.axis('off')
plt.subplot(122), plt.imshow(median_result, cmap='gray'), plt.title('After Median Filter'),
plt.axis('off')
plt.tight_layout()
plt.show()
```

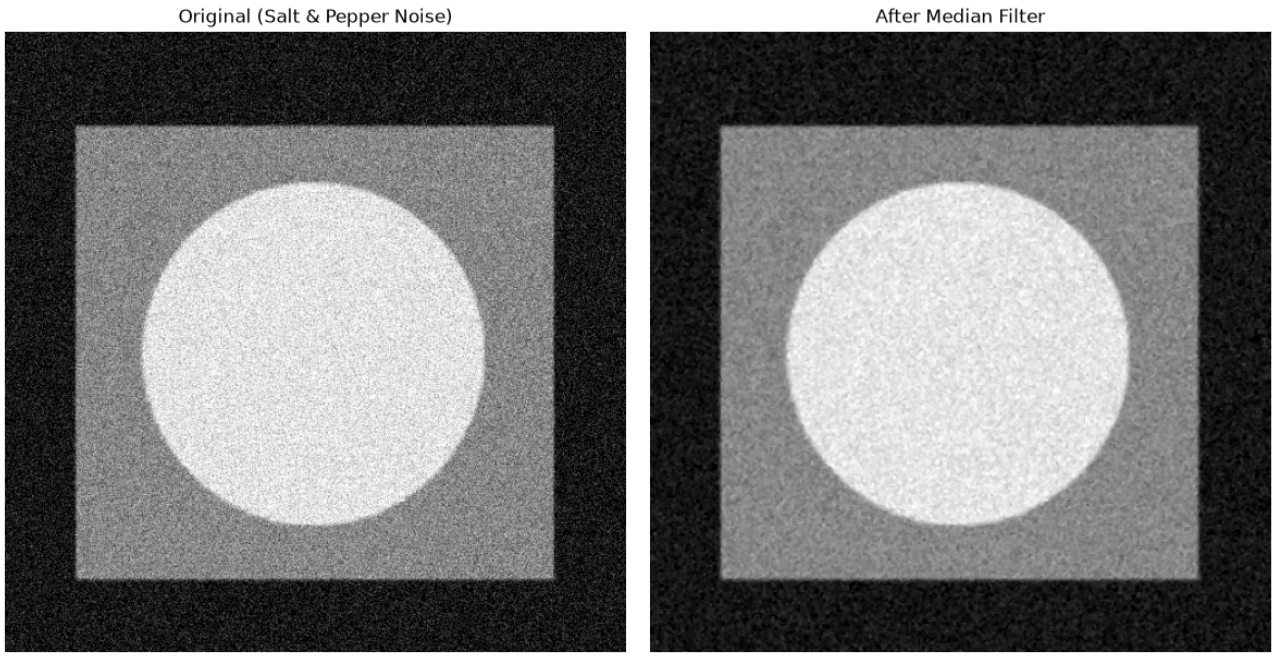


Figure 5: Gaussian-noise image (titled "Salt & Pepper Noise" in the notebook) and the result of the 3×3 median filter.

Discussion: The plot title says salt-and-pepper noise, but the input is again `noise_gauss`. The median filter reduces the grain while keeping the edges of the square and the disk sharp, because the median is always one of the existing pixel values and does not average across an edge. For Gaussian noise it was slightly worse than the arithmetic mean in the re-run (RMS error ≈ 10.5 compared with ≈ 8). The median filter is best suited to impulse noise. As an extra check, not part of the notebook, the same 3×3 median filter was applied to the salt-and-pepper image. It removed almost all of the impulses (PSNR ≈ 50 dB, compared with ≈ 27 dB for the 3×3 mean).

10. Max and Min Filters

The max filter outputs the brightest pixel in the window and the min filter outputs the darkest. They are implemented with the morphological operations `cv2.dilate` and `cv2.erode` using a 3×3 kernel of ones.

```
# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")
```

```
noise_img = noise_gauss

# 2. Define the Kernel size (e.g., 3x3)
kernel = np.ones((3, 3), np.uint8)

# 3. Apply Max Filter (Dilation)
# This picks the brightest pixel in the neighborhood.
max_filter = cv2.dilate(noise_img, kernel)

# 4. Apply Min Filter (Erosion)
# This picks the darkest pixel in the neighborhood.
min_filter = cv2.erode(noise_img, kernel)

# --- Display Results ---
plt.figure(figsize=(15, 5))

plt.subplot(131), plt.imshow(noise_img, cmap='gray'), plt.title('Original Noisy')
plt.subplot(132), plt.imshow(max_filter, cmap='gray'), plt.title('Max Filter (Removes
Pepper)')
plt.subplot(133), plt.imshow(min_filter, cmap='gray'), plt.title('Min Filter (Removes
Salt)')

for ax in plt.gcf().axes: ax.axis('off')
plt.tight_layout()
plt.show()
```

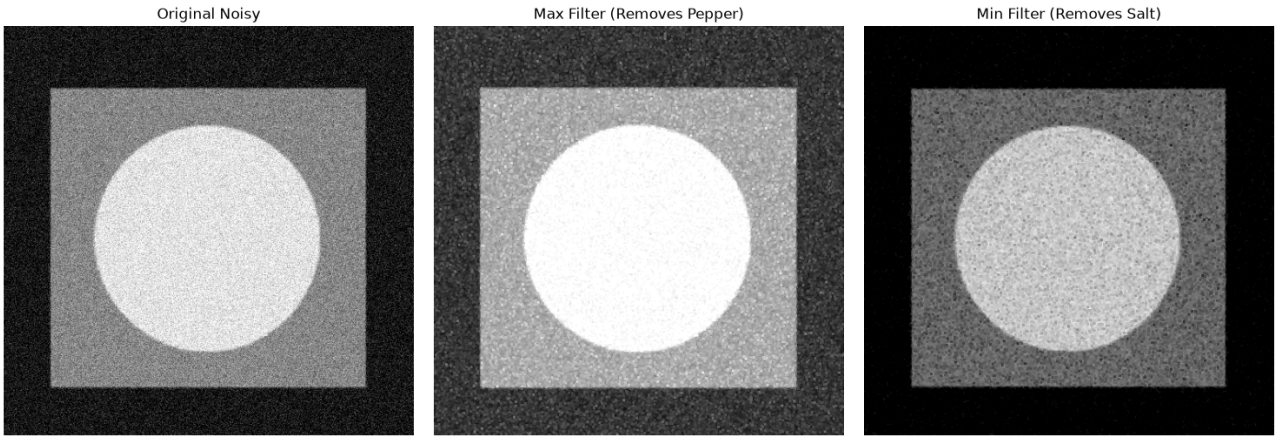


Figure 6: Gaussian-noise image and the results of the 3×3 max and min filters.

Discussion: The max filter makes the whole image brighter: the disk becomes almost pure white, the square and the background become lighter, and bright regions grow slightly. The min filter does the opposite: the background becomes black, the square and the disk become darker, and bright regions shrink slightly. In the re-run, the mean brightness changed by about +34 and −32 gray levels. With salt-and-pepper noise, the max filter would remove the black pepper dots and the min filter would remove the white salt dots, as the titles say. With Gaussian noise, where every pixel is disturbed, they simply move the image towards the extremes of the noise. These are the worst of all the filters in this lab for this noise type.

11. Midpoint Filter

The midpoint filter averages the maximum and minimum of the window:

$$f(x, y) = \tfrac{1}{2} [\max g(s, t) + \min g(s, t)]$$

It is computed from the dilated and eroded images in float, then converted back to 8-bit.

```
def midpoint_filter(image, kernel_size=3):
    # 1. Define the structural element (kernel)
    kernel = np.ones((kernel_size, kernel_size), np.uint8)

    # 2. Find the Maximum value in the neighborhood (Dilation)
    max_img = cv2.dilate(image, kernel).astype(np.float64)

    # 3. Find the Minimum value in the neighborhood (Erosion)
    min_img = cv2.erode(image, kernel).astype(np.float64)

    # 4. Calculate the average of Max and Min
    midpoint = (max_img + min_img) / 2

    # 5. Convert back to 8-bit
    return midpoint.astype(np.uint8)

# === List of noisy images
# noise_gauss = add_noise(img, "gaussian")
# noise_sp = add_noise(img, "salt_pepper")
# noise_expo = add_noise(img, "exponential")
# noise_uni = add_noise(img, "uniform")

noise_img = noise_gauss

# Apply the Midpoint Filter
result = midpoint_filter(noise_img, kernel_size=3)

# --- Display Results ---
plt.figure(figsize=(12, 6))

plt.subplot(121), plt.imshow(noise_img, cmap='gray'), plt.title('Original (Gaussian/Uniform
Noise)'), plt.axis('off')
plt.subplot(122), plt.imshow(result, cmap='gray'), plt.title('After Midpoint Filter'),
plt.axis('off')

plt.tight_layout()
plt.show()
```

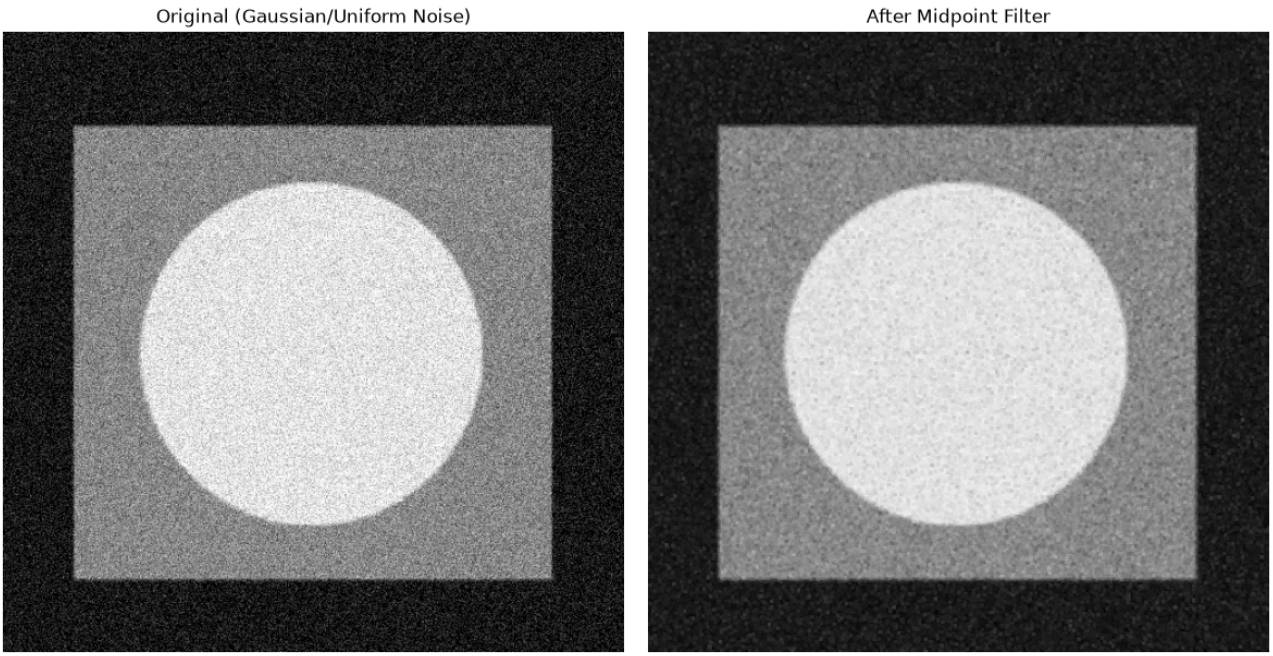


Figure 7: Gaussian-noise image and the result of the 3×3 midpoint filter.

Discussion: The midpoint filter combines order statistics with averaging and works best for randomly distributed noise such as Gaussian or uniform noise, which matches the input used here. The grain is clearly reduced while the overall brightness stays almost the same, because the brightening of the max and the darkening of the min cancel out. In the re-run, its RMS error (≈ 11) was close to that of the median filter. It is not suitable for salt-and-pepper noise, because one impulse in the window becomes the max or the min and pulls the result halfway towards 0 or 255.

12. Conclusion

The best filter depends on the type of noise. For the Gaussian noise used throughout this notebook, the arithmetic mean gave the lowest error, followed by the median and midpoint filters, which also preserve edges better. The geometric, harmonic and negative-Q contra-harmonic filters are pulled towards zero by dark pixels. They failed in the dark background, where the Gaussian noise had been clipped to 0. The max, min and positive-Q contra-harmonic filters shift the brightness and are only meant for one-sided impulse noise. Several plot titles in the notebook (salt-and-pepper, "Removes Pepper", "Removes Salt") describe what these filters are designed for, not the input they were actually given. For salt-and-pepper noise, the median filter is the clear choice, as the extra check showed.

13. References

R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Pearson – Chapter 5: Image Restoration.

Sandipan Dey, *Hands-On Image Processing with Python*, Packt.

OpenCV: Smoothing Images – docs.opencv.org/4.x/d4/d13/tutorial_py_filtering.html

OpenCV: Morphological Transformations – docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html